

Welcome back, John Smith

(DevOps)

Here's what's happening with your cloud infrastructure, deadlock graph, and security telemetry today.

Live
Refresh

+
Deploy
New
VM



Total Cloud Cores

32 Cores

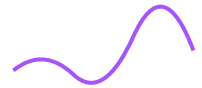


↗ +12.5% this month



Active Virtual Machines

5 / 6



↗ +8.2% this month



Security Precision Score

70.0%



↗ +2.1% this month



Deadlock Risk (WFG)

ALERT (3)

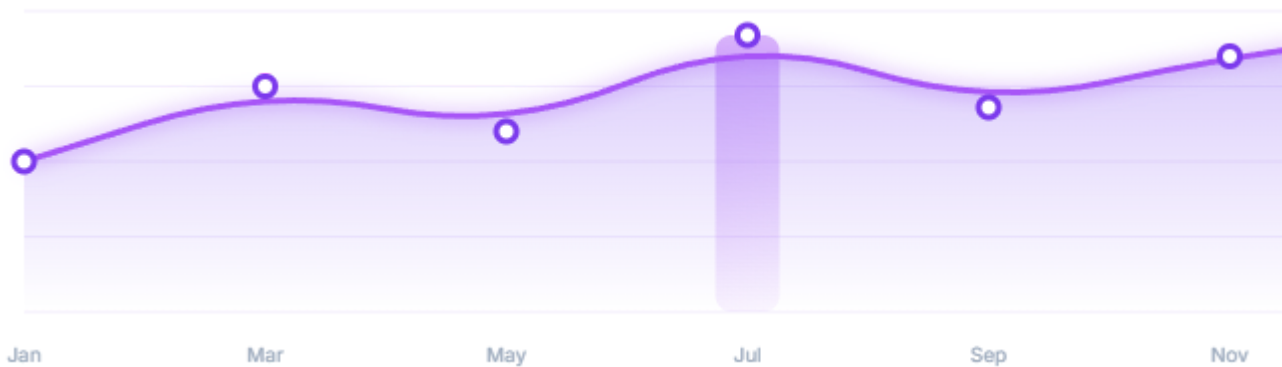


DFS Cycle Active

Cloud Infrastructure Performance Index

Auto Scale Active

Aggregated resource throughput, IOPS, and memory allocations over rolling quarters



Cyber Threat Index

Real-time security posture



Health 98% Load 24%

 Cloud Virtual Machines Management

Provision, inspect, scale, and monitor compute instances running on host cluster

+ Deploy Instance

 Cloud Hardware Resource Pools

Live tracking of physical CPU Cores, RAM, NVMe Storage, and Network Bandwidth

Pool Quotas Enforced

CPU

18.8% Used

6 Cores

of 32 Cores



Available: 26 Cores

Pool #1

RAM

43.8% Used

56 GB

of 128 GB



Available: 72 GB

Pool #2

STORAGE

25.0% Used

500 GB

of 2000 GB



Available: 1500 GB

Pool #3

NETWORK

30.0% Used

30 Gbps

of 100 Gbps



Available: 70 Gbps

Pool #4



Pending Resource Requests

+ New Request

Requests waiting for approval and ACID transactional allocation

ID	VM	RESOURCE	UNITS	STATUS	ACTIONS
#1	AI-Inference-Engine	CPU	8	REJECTED	REJECTED
#2	Backup-Vault-Server	RAM	32	ALLOCATED	ALLOCATED

 Active Resource Allocations

Live Allocations

Granted resource leases backed by PostgreSQL transactions

ALLOC ID	VM INSTANCE	RESOURCE POOL	LEASED UNITS	STATE	RELEASE
#19	Backup-Vault-Server	RAM	32	ACTIVE	<button>Release</button>
#12	Web-Frontend-Cluster	STORAGE	100	ACTIVE	<button>Release</button>
#13	Postgres-DB-Node	CPU	6	ACTIVE	<button>Release</button>
#14	Postgres-DB-Node	RAM	24	ACTIVE	<button>Release</button>
#15	Postgres-DB-Node	STORAGE	400	ACTIVE	<button>Release</button>



Deadlock Detection – Wait-For Graph (WFG) & DFS

Operating Systems cycle detection using 3-Color Depth-First Search algorithm

Simulate Cycle

Run DFS Detection

Auto-Resolve Deadlock

DEADLOCK DETECTED! Circular Wait Identified in Operating System Graph



VM-1 (Web-Frontend-Cluster) → VM-3 (AI-Inference-Engine) → VM-2 (Postgres-DB-Node) → VM-1 (Web-Frontend-Cluster)



ENTERPRISE CORE



Kernel Online

A

John Smith (DevOps)
OPERATOR



Dashboard



Virtual Machines



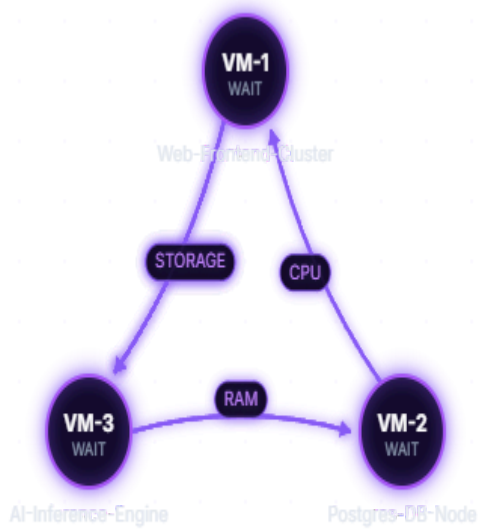
Resource Pools



Requests & Allocations



Deadlock WF



Interactive: Drag VM nodes to reposition graph

Acyclic Edge

Cycle (Deadlock)

Coffman Conditions for Deadlock

A deadlock occurs if and only if all 4 conditions hold: **Mutual Exclusion**, **Hold & Wait**, **No Preemption**, and **Circular Wait**.

Wait-For Graph (WFG) Principle

A simplified variant of Resource Allocation Graph (RAG) where vertices are processes and directed edge $\langle V_i \rightarrow V_j \rangle$ signifies that V_i is blocked waiting for V_j to release a resource.

3-Color DFS Cycle Algorithm

Vertices are labeled WHITE (unvisited), GRAY (on active recursion stack), or BLACK (visited). An edge encountered pointing to a GRAY vertex constitutes a back-edge, proving a directed cycle!

Automated Preemption Recovery

The resolution engine selects a victim VM in the cycle and breaks its wait dependency, preempting its allocation to restore the system to an acyclic safe state.

Cybersecurity Threat Detection & Radar

Active anomaly scanning for brute-force attacks, unauthorized privilege attempts, and resource abuse

Zero-Trust Guard

Live Incident Telemetry

Re-Scan

BRUTE_FORCE_LOCKOUT CRITICAL

Brute-force alert! User 'john' has been BLOCKED after 3 consecutive failed attempts.

Origin: 203.0.113.42 • Target: john

BRUTE_FORCE_LOCKOUT CRITICAL

Brute-force alert! User 'john' has been BLOCKED after 3 consecutive failed attempts.
Origin: 203.0.113.42 • Target: john

Cybersecurity Threat Simulation & Testing

Test defensive triggers and automatic account lockout mechanisms for viva presentation:

Simulate Brute-Force Attack

Simulate Resource Abuse Spike

Unblock DevOps Account (John)

Cybersecurity Defense Mechanisms Implemented:

- **Salted SHA-256 Hashing:** Cryptographic password hashing to prevent credential theft.
- **PreparedStatement Defense:** Parameterized SQL queries preventing SQL injection across all DAOs.
- **Brute-Force Lockout:** Accounts are locked after 3 consecutive failed logins within 5 minutes.
- **Role-Based Access Control (RBAC):** Granular permissions separating Admin, Operator, and Viewer.
- **Audit Logging:** Centralized security event logging with severity levels (LOW, MEDIUM, HIGH, CRITICAL).



System & Security Audit Logs

Refresh Logs

Immutable audit trail of authentications, security violations, and deadlock incidents

Security Logs

Login Logs

Deadlock Logs

ID	USERNAME	IP ADDRESS	STATUS	REASON	ATTEMPT TIME
#25	john	0:0:0:0:0:0:1	SUCCESS	N/A	2026-09-06 21:48:33.261
#24	admin	0:0:0:0:0:0:1	SUCCESS	N/A	2026-09-06 21:45:56.611
#23	john	0:0:0:0:0:0:1	SUCCESS	N/A	2026-09-06 21:45:44.046
#22	admin	0:0:0:0:0:0:1	SUCCESS	N/A	2026-09-06 21:45:37.977
#21	john	0:0:0:0:0:0:1	BLOCKED	Account is locked	2026-09-06 21:45:34.3

ID	USERNAME	IP ADDRESS	STATUS	REASON	ATTEMPT TIME
#20	admin	0:0:0:0:0:0:1	SUCCESS	N/A	2026-09-06 21:45:21.111
#19	john	203.0.113.42	BLOCKED	Account is locked	2026-09-06 21:45:04.758
#18	john	203.0.113.42	BLOCKED	Account is locked	2026-09-06 21:45:04.758
#17	john	203.0.113.42	BLOCKED	Account is locked	2026-09-06 21:45:04.758
#16	alex	0:0:0:0:0:0:1	SUCCESS	N/A	2026-09-06 21:44:35.81
#15	john	0:0:0:0:0:0:1	BLOCKED	Account is locked	2026-09-06 21:44:31.895
#14	john	0:0:0:0:0:0:1	BLOCKED	Account is locked	2026-09-06 21:44:23.796
#13	admin	0:0:0:0:0:0:1	SUCCESS	N/A	2026-09-06 21:44:16.667
#12	john	203.0.113.42	BLOCKED	Password mismatch	2026-09-06 21:43:52.02
#11	john	203.0.113.42	FAILED	Password mismatch	2026-09-06 21:43:52.02
#10	john	203.0.113.42	FAILED	Password mismatch	2026-09-06 21:43:52.02
#9	john	0:0:0:0:0:0:1	SUCCESS	N/A	2026-09-06 21:43:10.98
#8	admin	0:0:0:0:0:0:1	SUCCESS	N/A	2026-09-06 21:43:04.329
#7	john	203.0.113.42	BLOCKED	Password mismatch	2026-09-06 21:38:22.726
#6	john	203.0.113.42	FAILED	Password mismatch	2026-09-06 21:38:22.726
#5	john	203.0.113.42	FAILED	Password mismatch	2026-09-06 21:38:22.723

ID	USERNAME	IP ADDRESS	STATUS	REASON	ATTEMPT TIME
#1	admin	192.168.1.10	SUCCESS	N/A	2026-09-06 21:14:07.5
#2	john	192.168.1.25	SUCCESS	N/A	2026-09-06 21:14:07.5
#3	alex	192.168.1.42	SUCCESS	N/A	2026-09-06 21:14:07.5
#4	guest	192.168.1.99	FAILED	User does not exist	2026-09-06 21:14:07.5



Analytical Reports & PostgreSQL Views

Print / Export PDF

High-level summaries aggregated directly from PostgreSQL database views

Database Schema
PostgreSQL
18
9 Tables • 5 Views •
3 Triggers

Security Incidents
(24h)
4 Logged
View:
view_security_incident_rep

Average CPU
Usage
43.7%
View:
view_resource_utilization

Active WFG Edges
3 Edges
View:
view_active_wait_graph

SQL View Definition Reference (04_views.sql):

```
CREATE OR REPLACE VIEW view_system_kpis AS
SELECT
  (SELECT COUNT(*) FROM virtual_machines) AS total_vms,
  (SELECT COUNT(*) FROM virtual_machines WHERE status = 'RUNNING') AS running_vms,
  (SELECT COUNT(*) FROM users) AS total_users,
  (SELECT COUNT(*) FROM deadlock_logs WHERE resolution_status = 'DETECTED') AS unresolved_deadlocks,
  (SELECT COUNT(*) FROM security_logs WHERE severity IN ('HIGH', 'CRITICAL')) AS critical_threats;
```



BCA Viva & Academic Architecture Reference Guide

BCA Project Viva

Key viva questions, concepts, and architectural explanations for project defense

 **OOP & Architecture (Model-DAO-Service-Controller)**

Q: Explain the Model-DAO-Service-Controller architecture used in CloudShield.

- **Model:** Pure Java POJO beans (User, Resource, VirtualMachine) encapsulating entities with private attributes and getters/setters.
- **DAO:** Data Access Objects (UserDAO, AllocationDAO) using JDBC PreparedStatements for safe database operations.
- **Service:** Business logic layer (DeadlockDetector, ThreatDetector, SecurityManager) implementing algorithms independently of UI.
- **Controller:** REST API controller (ApiController, CloudShieldServlet) dispatching JSON HTTP requests.

Q: How are OOP principles applied in this project?

- **Encapsulation:** Private variables in models accessed solely through validated getters/setters.
- **Abstraction:** Services hide complex graph algorithms (DFS) and cryptographic hashing behind clean method signatures.
- **Single Responsibility:** Each DAO manages exactly one database table.

Operating Systems & Deadlock Detection
Q: What is a Wait-For Graph (WFG) and how is it used here?

In OS theory, a Wait-For Graph is a directed graph where vertices are processes (Virtual Machines) and edges denote that process A is waiting for a resource held by process B. For single-unit resources, a cycle in the WFG is both a necessary and sufficient condition for a deadlock.

Q: How does the DFS Cycle Detection algorithm work?

We implement a 3-Color DFS traversal (WHITE/GRAY/BLACK). When an edge is traversed to an already GRAY vertex (currently on the recursion call stack), a back-edge is identified, proving the existence of a circular wait condition.

Cybersecurity & Threat Defense
Q: How does CloudShield protect against SQL Injection?

All database queries use PreparedStatement with parameter placeholders (?). The database driver treats user input strictly as literal values rather than executable SQL syntax.

Q: How is brute-force authentication prevented?

Every login is tracked in login_logs. If 3 consecutive failed login attempts occur, the account is automatically locked (account_status = 'BLOCKED') and a CRITICAL alert is logged.

Database & PostgreSQL Features
Q: How are ACID transactions implemented in Java JDBC?

In AllocationDAO.java, we invoke connection.setAutoCommit(false). We lock the resource row (FOR UPDATE), verify availability, deduct balance, insert allocation, and invoke connection.commit(). If any failure occurs, connection.rollback() restores consistency.

Q: What database constraints and triggers are utilized?

- **CHECK constraints:** Enforce non-negative resource balances and valid statuses.
- **FOREIGN KEYS:** Ensure referential integrity with cascading deletes.
- **TRIGGERS:** Automated lockout trigger (`trg_login_security`) and VM state sync.